

11 — Phân tích hợp nhất 3 nhánh & Đề xuất kế hoạch merge tối ưu

Người tổng hợp: Tăng Huỳnh Tuấn Tú (Authentication + Employee) **Ngày:** 20/07/2026 · **Nhánh cơ sở dùng để đối chiếu:** TuanTu **Cách đọc tài liệu này:** viết theo góc nhìn phân tích trung lập kiểu PM — đối chiếu **3 nhánh cùng lúc** (Tuấn Tú / Long Nguyễn / Minh Anh), không phải báo cáo 1 chiều nghiêng về 1 người. Mỗi kết luận đều kèm bằng chứng kỹ thuật cụ thể (code, hành vi thực tế khi merge), không dựa trên ai "nên" thắng.

File này **tổng hợp** 09-xung-dot-longnguyen-va-de-xuat-merge.md và 10-xung-dot-minhanh-va-de-xuat-merge.md thành 1 bức tranh 3 nhánh duy nhất — 2 file gốc vẫn giữ nguyên để tra cứu chi tiết từng cặp nhánh riêng lẻ. Giá trị thêm của file này: gộp các phát hiện **trùng lặp giữa Long và Minh Anh** (để không phải đọc 2 lần cùng 1 vấn đề), và đưa ra **1 trình tự merge duy nhất** xử lý cả 3 nhánh cùng lúc thay vì làm 2 lần đọc lặp.

Phương pháp kiểm tra: git merge-tree --write-tree (mô phỏng merge thật, không sửa gì) để lấy danh sách conflict, sau đó merge thật trên 2 nhánh test cục bộ, không commit (test_merge1 cho Long, test_merge_minhanh trong 1 worktree riêng cho Minh Anh) để đọc đúng nội dung từng bên.

Phần 1 — Bức tranh tổng thể: mỗi file, ai có code gì

File	Tuấn Tú	Long	Minh Anh	Mức độ	Ghi chú nhanh
accounts/authentication.py	✓	✓	✓	🔴	Long & Minh Anh dùng chung 1 bản (cùng gốc), cả 2 đều thiếu blacklist logout mà Tuấn Tú đã có
accounts/models.py (CustomUser)	✓	✓	✓	🟡	Tưởng là 3 thiết kế khác nhau, hoá ra Long & Minh Anh chung gốc — thực chất chỉ 2 thiết kế cần đối chiếu
worktracker_core/settings.py	✓	✓	✓	🔴	Cả 2 bản kia đều làm mất SIMPLE_JWT/Email/Redis-blacklist/CORS nếu lấy nguyên
worktracker_core/urls.py	✓	✓	✓	🔴	Cả 3 người dùng 3 quy ước tiền tố route khác nhau
requirements.txt	✓	✓	✓	🟢	Trivial, union lấy version mới nhất
accounts/permissions.py	✓	✗	✓	🔴🔴 Nặng nhất toàn bộ investigation	Chỉ Minh Anh viết đề — nhưng là lỗi kỹ thuật (không tương thích), không phải khác gu
accounts/urls.py / views.py	(rỗng, chủ đích)	✗	✓	🔴	Chỉ Minh Anh có code ở đây — vi phạm convention file đã thống nhất từ đầu
timesheets/urls_manager.py	✓ (26 dòng)	✓ (Router-based)	✗	🔴	Chỉ Long có bản đầy đủ log-works + time-locks
timesheets/views_manager.py	✓ (26 dòng)	✓ (463 dòng)	✗	🔴 nặng nhất về khối lượng	Long: review/approve/reject/correct/void/unlock đầy đủ
timesheets/serializers_manager.py	✓ (1 serializer)	✓ (~15 serializer)	✗	🔴	Chỉ Long

Quan sát quan trọng nhất: 4 trong 5 file nặng nhất (authentication.py, models.py, settings.py, urls.py core) bị đụng bởi **cả 3 nhánh, với cùng nguyên nhân gốc ở 2/3** — vì Long và Minh Anh đã thống nhất kiến trúc với nhau từ trước (commit ec87153 "Merge branch 'LongNguyen' ... into MinhAnh"), độc lập với nhánh Tuấn Tú. Hệ quả thực dụng: **giải quyết những file này 1 lần là đủ cho cả 2 phía kia**, không cần làm 2 lần riêng biệt.

Phần 2 — Phát hiện xuyên suốt (gộp theo chủ đề, không lặp lại theo từng nhánh)

2.1 — accounts/authentication.py: Long và Minh Anh dùng chung 1 thiết kế, cả 2 đều thiếu 1 tầng

Cả 2 bên đều là class CachedIsActiveJWTAuthentication, logic giống hệt nhau (bản Minh Anh chỉ thêm comment tiếng Việt giải thích từng dòng — cùng 1 tác giả gốc). Cả 2 đều **thiếu lớp blacklist logout** mà nhánh Tuấn Tú đã có (BlacklistAwareJWTAuthentication) — nghĩa là nếu 1 trong 2 bản này thắng, token vẫn dùng được sau khi user bấm logout.

Nhánh Tuấn Tú (accounts/authentication.py):

```
from django.core.cache import cache
from rest_framework.exceptions import AuthenticationFailed
from rest_framework_simplejwt.authentication import JWTAuthentication
from .redis_client import redis_client

# Rejects any token whose jti has been blacklisted in Redis after logout.
class BlacklistAwareJWTAuthentication(JWTAuthentication):
    def get_validated_token(self, raw_token):
        validated_token = super().get_validated_token(raw_token)
        jti = validated_token["jti"]
        if redis_client.exists(f"blacklist:{jti}"):
            raise AuthenticationFailed("Token has been revoked.")
        return validated_token

_ACTIVE_CACHE_PREFIX = "user_active:"
_ACTIVE_CACHE_TTL = 300

def get_user_active_status(user_id):
    cache_key = f"{_ACTIVE_CACHE_PREFIX}{user_id}"
    cached = cache.get(cache_key)
    if cached is not None:
        return cached
    from accounts.models import CustomUser
    try:
        is_active = CustomUser.objects.values_list("is_active", flat=True).get(pk=user_id)
    except CustomUser.DoesNotExist:
        return False
    cache.set(cache_key, is_active, timeout=_ACTIVE_CACHE_TTL)
    return is_active

def set_user_active_status(user_id, is_active):
    cache.set(f"{_ACTIVE_CACHE_PREFIX}{user_id}", is_active, timeout=_ACTIVE_CACHE_TTL)

def invalidate_user_active_status(user_id):
    cache.delete(f"{_ACTIVE_CACHE_PREFIX}{user_id}")

# Extends BlacklistAwareJWTAuthentication with an is_active check via Redis cache.
# This is the class used in DEFAULT_AUTHENTICATION_CLASSES.
class WorkTrackerJWTAuthentication(BlacklistAwareJWTAuthentication):
    def authenticate(self, request):
        result = super().authenticate(request)
        if result is None:
            return None
        user, validated_token = result
        if not get_user_active_status(user.id):
            raise AuthenticationFailed(
                "Account is locked or deactivated.", code="account_inactive"
            )
        return user, validated_token
```

Nhánh Long / Minh Anh (chung, chỉ khác comment):

```
class CachedIsActiveJWTAuthentication(JWTAuthentication):
    def authenticate(self, request):
        result = super().authenticate(request)
        if result is None:
            return None
        user, validated_token = result
        if not get_user_active_status(user.id):
            raise AuthenticationFailed(
                "Account is locked or deactivated.", code="account_inactive"
            )
        return user, validated_token
```

— thiếu hẳn phần tương đương BlacklistAwareJWTAuthentication.

Đánh giá: nhánh Tuấn Tú là bản duy nhất còn giữ được tính năng logout thu hồi token ngay lập tức. Khuyến nghị dùng làm nền, không có phương án thay thế nào từ 2 nhánh kia bù lại được phần đã thiếu.

Ghi chú riêng (không ảnh hưởng quyết định merge, phát hiện khi review kỹ thuật riêng): lớp `is_active` cache ở cả 3 bên thực ra là `dead weight` — `JWTAuthentication.get_user()` gốc của `SimpleJWT` đã tự check `is_active` bằng query DB tươi mỗi request rồi (`CHECK_USER_IS_ACTIVE` default `True`, không ai override). Không ảnh hưởng merge, đáng dọn sau khi mọi thứ ổn định.

2.2 — `accounts/models.py` (`CustomUser`): thực chất chỉ có 2 thiết kế, không phải 3

So trực tiếp `origin/LongNguyen:accounts/models.py` với `origin/MinhAnh:accounts/models.py` — chỉ khác 4 dòng (`Role.is_active`, `Permission.group`, `EmployeeProfile.joined_date`, 1 chữ trong comment). Bản của Minh Anh = bản của Long (thừa hưởng qua `ec87153`) + 3 field nhỏ riêng của cô ấy. Vậy thực chất cuộc so sánh chỉ còn **2 phía**: Tuấn Tú vs. (Long + Minh Anh).

Nhánh Tuấn Tú:

```
class CustomUser(AbstractUser):
    first_name = None
    last_name = None

    email = models.EmailField(max_length=155, unique=True, db_index=True)
    role = models.ForeignKey(
        Role, on_delete=models.RESTRICT, null=True
    )

    # is_active đã có sẵn trong AbstractUser (chuyển FALSE khi Admin khóa/nghi việc)
    must_change_password = models.BooleanField(
        default=True
    ) # Cờ buộc đổi mật khẩu lần đầu đăng nhập hoặc sau khi reset
    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["username"] # ⚠️ xem ghi chú bug bên dưới

    def __str__(self):
        return self.email
```

Nhánh Long (Minh Anh giống hệt + 3 field nhỏ):

```
class CustomUser(AbstractUser):
    # ... (email, role giống nhánh Tuấn Tú)

    is_active = models.BooleanField(default=True, db_index=True) # khai lại để thêm db_index

    objects = CustomUserManager() # validate bắt buộc có role khi tạo superuser

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["role"] # role là FK bắt buộc

    class Meta:
        db_table = "users"
```

Đối chiếu điểm mạnh/yếu 2 bên:

- Nhánh Tuấn Tú có `must_change_password` — tính năng Giai đoạn 5 (account lifecycle) phụ thuộc trực tiếp field này, không có bên nào khác có.
- Nhánh Long có `CustomUserManager` (validate role bắt buộc khi tạo superuser — an toàn hơn), `Meta.db_table="users"` (đặt tên bảng rõ ràng, nhánh Tuấn Tú đang thiếu, để mặc định `accounts_customuser`), và `REQUIRED_FIELDS=["role"]` đúng hơn (xem bug bên dưới).

→ **Đề xuất:** hợp nhất — giữ `must_change_password` (chỉ có ở Tuấn Tú) + lấy

`CustomUserManager/Meta.db_table/REQUIRED_FIELDS=["role"]` (đúng hơn, từ Long) + cộng thêm 3 field nhỏ của Minh Anh. Không phải "chọn phe" — đa số phần đều tương thích, gộp được vào 1 thiết kế duy nhất qua 1 migration mới.

Bug có sẵn phát hiện tình cờ khi đối chiếu (không do merge gây ra): nhánh Tuấn Tú có `REQUIRED_FIELDS = ["username"]` trong khi `username = None` đã bị xóa khỏi `CustomUser` cùng class — `createsuperuser` sẽ lỗi vì cố hỏi 1 field không tồn tại. `REQUIRED_FIELDS=["role"]` của nhánh Long đúng hơn, nên lấy khi viết migration hợp nhất.

2.3 — `accounts/permissions.py`: phát hiện nghiêm trọng nhất toàn bộ investigation (chỉ Minh Anh đụng)

Long không dùng file này (dùng `system.permissions_manager` riêng cho phần của mình). Chỉ nhánh Minh Anh viết đề `HasPermission`:

```
# Nhánh Minh Anh – nhận tham số lúc khởi tạo
class HasPermission(BasePermission):
    def __init__(self, required_permission):
        self.required_permission = required_permission

    def has_permission(self, request, view):
        user = request.user
        if not user or not user.is_authenticated:
            return False
        if not hasattr(user, 'role') or user.role is None:
            return False
        return user.role.role_permissions.filter(permission__code=self.required_permission).exists()
```

Cách gọi tương ứng bên `accounts/views.py` của Minh Anh: `HasPermission('user:create')` — khởi tạo có tham số.

Nhánh Tuấn Tú:

```
from rest_framework.permissions import BasePermission
from rest_framework.exceptions import PermissionDenied
from .models import RolePermission

class HasPermission(BasePermission):
    def has_permission(self, request, view):
        required_code = getattr(view, "required_permission", None)

        if required_code is None:
            raise AssertionError(
                f"{view.__class__.__name__} is missing a 'required_permission' "
                "attribute. Set it to the permission code this view requires "
                "(e.g. 'client:create').")

        if not request.user or not request.user.is_authenticated:
            return False

        if request.user.must_change_password:
            raise PermissionDenied("You must change your password before performing this action.")

        if request.user.role is None:
            return False

        return RolePermission.objects.filter(
            role=request.user.role, permission__code=required_code
        ).exists()
```

Cách gọi tương ứng — convention chuẩn của DRF, khởi tạo 0 tham số:

```
class AdminDisableUserView(APIView):
    permission_classes = [HasPermission]
    required_permission = "user:disable"
```

Đánh giá kỹ thuật (không phải chuyện gu thiết kế): DRF luôn khởi tạo permission class bằng constructor **0 tham số** (`permission_classes = [HasPermission]` → DRF tự gọi `HasPermission()`). Toàn bộ view hiện có trong hệ thống — của Tuấn Tú, và cả `ManagerLogWorkViewSet` của Long — đều dựa vào convention này. Nếu bản của Minh Anh (constructor có tham số bắt buộc) được merge vào vị trí `accounts.permissions.HasPermission`, **mọi view dùng đúng convention chuẩn sẽ crash** `TypeError: __init__() missing 1 required positional argument` ngay khi DRF khởi tạo — không phải lỗi riêng của Minh Anh, mà là lỗi sập toàn hệ thống.

→ **Kết luận khách quan:** đây không phải trường hợp "2 thiết kế hợp lý, chọn 1" — bản của Tuấn Tú là bản **duy nhất tương thích** với phần còn lại của codebase đang tồn tại. Nếu Minh Anh cần permission theo tham số động ở vài chỗ đặc biệt, giải pháp đúng là thêm 1 class mới (ví dụ đặt tên `HasPermissionCode`, tương tự cách đặt tên bên `system.permissions_manager` của Long — nên tham khảo cả 2 để thống nhất quy ước) — không sửa `HasPermission` gốc.

2.4 — `worktracker_core/settings.py`: cùng 1 lỗi lặp lại ở cả 2 nhánh còn lại

Cả nhánh Long và Minh Anh, nếu lấy nguyên bản, đều làm mất: SIMPLE_JWT (lifetime/rotation), EMAIL_BACKEND (forgot-password vỡ), REDIS_BLACKLIST_DB (code redis_client.py ném AttributeError), CORS_ALLOWED_ORIGINS (Frontend Vite dev bị chặn CORS hoàn toàn).

Nhánh Tuấn Tú (khối cần giữ):

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "accounts.authentication.WorkTrackerJWTAuthentication",
    ),
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
}

from datetime import timedelta

SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "ROTATE_REFRESH_TOKENS": True,
    "BLACKLIST_AFTER_ROTATION": True,
}

EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
DEFAULT_FROM_EMAIL = 'no-reply@worktracker.com'

# db=1: JWT blacklist after logout – uses redis_client directly.
REDIS_HOST = "127.0.0.1"
REDIS_PORT = 6379
REDIS_BLACKLIST_DB = 1

# db=2: is_active cache.
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
        "LOCATION": f"redis://{REDIS_HOST}:{REDIS_PORT}/2",
    }
}

CORS_ALLOWED_ORIGINS = [
    "http://localhost:5173",
]
```

Khác biệt giữa Long và Minh Anh (thông tin phụ): Long dùng django_redis (db=1) cho is_active cache, Minh Anh dùng LocMemCache (cache trong RAM của từng process — yếu hơn cả Long, không chia sẻ được giữa các worker khi chạy đa tiến trình, ví dụ gunicorn). Không ảnh hưởng tới quyết định chính vì cả khối REST_FRAMEWORK/CACHES của cả 2 bên đều bị loại bỏ để giữ bản Tuấn Tú.

Điểm hay đáng cân nhắc lấy đọc lập (không phải một phần bắt buộc của merge): khối DATABASES của Minh Anh đọc toàn bộ field từ biến môi trường (os.environ.get('DB_NAME'), DB_USER, DB_HOST, DB_PORT...), trong khi nhánh Tuấn Tú chỉ đọc PASSWORD từ .env, còn lại hardcoded. Cách làm của Minh Anh portable hơn khi đổi máy/deploy — đáng áp dụng bất kể ai thắng phần còn lại.

2.5 — worktracker_core/urls.py: 3 người, 3 quy ước tiền tố khác nhau

Cả Long và Minh Anh đều bypass LoginView bằng TokenObtainPairView/TokenRefreshView gốc của SimpleJWT — mất custom claims (role/email trong JWT), mất message chống dò email, mất user/permissions payload mà Frontend đang phụ thuộc. Nhưng vấn đề lớn hơn nằm ở tiền tố route:

Nhánh	Tiền tố đang dùng
Tuấn Tú	api/auth/...
Long	api/manager/...
Minh Anh	api/v1/auth/..., api/projects/..., api/accounts/..., api/system/...

Nhánh Tuấn Tú (route đầy đủ):

```

from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),

    # ===== AUTH =====
    path('api/auth/', include('accounts.urls_auth')),

    # ===== ADMIN =====
    path('api/auth/', include('accounts.urls_admin')),

    # ===== MANAGER =====
    path('api/auth/', include('accounts.urls_manager')),

    # ===== TIMESHEETS =====
    path('api/timesheets/', include('timesheets.urls_manager')),
    path('api/timesheets/', include('timesheets.urls_employee')),
]

```

Đánh giá: không có nhánh nào "đúng tuyệt đối" ở khoản tiền tố — cả 3 người tự chọn quy ước riêng khi làm việc song song, không có chuẩn chung được thống nhất từ đầu. Đây là việc **cả 3 người cần ngồi lại quyết định cùng nhau 1 lần cho xong**, không phải chuyển kỹ thuật đơn thuần — càng để lâu càng khó đổi vì Frontend sẽ bắt đầu code cứng theo 1 trong 3 kiểu. Route LoginView (giữ custom claims + anti-enumeration + user payload) nên giữ theo nhánh Tuấn Tú vì đây là phần đã hoàn thiện và được Frontend Auth Kit (04-frontend-auth-kit.md) dựa vào.

2.6 — Kiến trúc app tổng thể: Long + Minh Anh dùng chung, Tuấn Tú dùng riêng

Nhánh Tuấn Tú: `accounts / clients / jobs / audit / notifications / timesheets`. Nhánh Long + Minh Anh (chung, qua ec87153): `accounts / projects / tasks / reports / system / timesheets`.

Đã verify: `clients/jobs/audit/notifications` bên nhánh Tuấn Tú vẫn là scaffold rỗng (mỗi `models.py` chỉ 3 dòng, chưa có model thật) — nghĩa là chuyển sang dùng `projects/system/reports` làm nền chính thức **không làm mất việc thật nào của bất kỳ ai**. Trong tất cả các quyết định kiến trúc cần chốt, đây là quyết định có chi phí thấp nhất.

2.7 — `timesheets/*_manager.py`: chỉ Long có code, model đã sẵn sàng, trả lời luôn FR-124

Minh Anh không đụng tới `timesheets` chút nào. Long đã code đầy đủ `ManagerLogWorkViewSet` (`list/retrieve/approve/reject/correct/void`) + `ManagerTimeLockViewSet` (`list/retrieve/create/unlock`) — 463 dòng, dùng Router + service layer riêng (`timesheets.services.logwork_review_manager_service`, `timesheets.services.time_lock_manager_service`). Nhánh Tuấn Tú chỉ có `ManagerTimeLockView` (26 dòng, APIView thuần, chỉ làm chiều khóa số).

Đã verify `timesheets/models.py` (tự merge sạch, không conflict) có sẵn đủ field mà code của Long cần (`review_status`, `reviewed_by/at/note`, `adjusted_by/at/reason`, `lock_scope`, `job`, `unlocked_by/at`, `unlock_reason`) — đúng là 2 migration mà [05-cho-minh-anh.md](#) đang chờ, nhánh Long đã làm tương đương sẵn. → Trả lời luôn câu hỏi FR-124 đang treo ở [00-tong-quan.md](#): phần `review/approve/reject/void` đúng là phạm vi của Long theo v2, không phải Tuấn Tú code trùng.

Đánh giá: đây là trường hợp hiếm hoi có thể so sánh trực tiếp mức độ hoàn thiện — bản của Long bao phủ toàn bộ workflow, bản của Tuấn Tú chỉ là một phần nhỏ (time-lock). Đề xuất dùng bản Long làm nền cho toàn bộ Manager-side timesheet.

2.8 — `accounts/urls.py` + `views.py`: chỉ Minh Anh có code, vi phạm convention file nhưng logic nghiệp vụ ổn

Minh Anh viết `UserViewSet/RoleViewSet/PermissionViewSet/DepartmentViewSet` — logic nghiệp vụ nhìn chung ổn (gọi đúng `set_user_active_status` khi lock/unlock, tách `UserCreateSerializer` riêng cho action create, `select_related` hợp lý) — nhưng đặt trong file default `accounts/views.py/accounts/urls.py` thay vì `views_admin.py/urls_admin.py/serializers_admin.py` đã scaffold sẵn cho vai trò Admin, đúng như nghi ngờ ban đầu ghi trong [00-tong-quan.md](#). Route đăng ký ở `/api/accounts/`, khác với convention `/api/auth/user/...` bên Tuấn Tú.

Đánh giá: phần logic không cần viết lại — chỉ cần di chuyển đúng file và đổi cách dùng `HasPermission` sang class-attribute style (2.3).

2.9 — `requirements.txt`: trivial ở cả 2 phía

Long thêm `django-redis/openpyxl/et_xmlfile`, bump `django-simple-history` 3.11→3.12 và `redis` 8.0.0→8.0.1. Minh Anh có version cũ hơn ở 2 gói trùng (`django-simple-history` 3.8.0, `django-rest-framework-simplejwt` 5.5.0) so với nhánh Tuấn Tú — không cần lấy gì từ Minh Anh ở file này, chỉ cần union với phần thêm của Long.

Phần 3 — Bảng quyết định hợp nhất (toàn bộ file, 1 lần duy nhất)

File	Đề xuất	Cơ sở đề xuất	Việc cần làm
accounts/authentication.py	Dùng bản Tuấn Tú	Bản duy nhất có lớp blacklist logout (2.1)	—
accounts/permissions.py	Dùng bản Tuấn Tú	Bản duy nhất tương thích kỹ thuật với DRF convention toàn hệ thống (2.3)	Minh Anh viết nếu cần permission số
accounts/models.py (CustomUser)	Hợp nhất 3 bên	must_change_password (Tuấn Tú) + CustomUserManager/db_table/REQUIRED_FIELDS (Long) + 3 field nhỏ (Minh Anh) (2.2)	1 migration mới
accounts/migrations/	Viết lại tuyến tính	Đang có 2 file 0002 khác nội dung song song	Không giữ song
accounts/urls.py, accounts/views.py	Giữ file default rỗng	Đúng convention đã thống nhất từ đầu (2.8)	Minh Anh chỉ ViewSet san _admin.py, dùng HasPe
worktracker_core/settings.py	Dùng khối JWT/Redis/Email/CORS của Tuấn Tú	Bản duy nhất còn đủ cấu hình, không làm vỡ tính năng nào (2.4)	INSTALLED_ 'reports'; DATABASES Minh Anh, k bị buộc)
worktracker_core/urls.py	Dùng route /api/auth/... của Tuấn Tú làm nền	Route duy nhất giữ được LoginView custom (2.5)	Chờ team ch ước prefix, r route Manag Admin (Minh song
timesheets/urls_manager.py	Dùng bản Long	Đầy đủ hơn nhiều (2.7)	Xoá ManagerTin cũ
timesheets/views_manager.py	Dùng bản Long	463 dòng vs 26 dòng, bao phủ toàn bộ workflow (2.7)	Xác nhận sy đã merge đủ có)
timesheets/serializers_manager.py	Dùng bản Long	Model đã có đủ field hỗ trợ (2.7)	Không cần s
requirements.txt	Union, lấy version mới nhất	Trivial (2.9)	pip instal requirem manage.py

Phần 4 — Kịch bản merge tối ưu (1 trình tự cho cả 3 nhánh, không làm 2 lần)

Nguyên lý tối ưu: vì phần lớn conflict nặng nhất **giống hệt nhau giữa Long và Minh Anh** (Phần 1), nên thay vì merge từng nhánh rồi resolve lại từ đầu 2 lần, làm **một bước chuẩn bị chung trước** trên nhánh cơ sở, để khi merge thật, git tự thấy hunck đã giống nhau và không còn báo conflict ở những chỗ đã xử lý.

Bước 0 — Chuẩn bị trên nhánh TuanTu (làm 1 lần, phục vụ cả 2 merge sau)

1. Viết 1 migration mới trong accounts (sau 0004): thêm Meta.db_table="users", đổi REQUIRED_FIELDS=["role"], giữ must_change_password, cộng 3 field nhỏ của Minh Anh (Role.is_active, Permission.group, EmployeeProfile.joined_date).

2. Union requirements.txt với phần thêm của Long (django-redis, openpyxl, et_xmlfile, version bump) — bỏ qua phần của Minh Anh (cũ hơn).

3. Không sửa authentication.py/permissions.py/khối JWT trong settings.py — đây đã là bản đề xuất giữ nguyên.

4. Commit riêng bước chuẩn bị này.
- Sau bước này, khi chạy git merge origin/LongNguyen hoặc git merge origin/MinhAnh thật, phần models.py/authentication.py gần như không còn conflict thật sự nữa (vì phía Tuấn Tú đã là superset), tiết kiệm đáng kể thời gian resolve so với làm riêng lẻ từng nhánh.

Bước 1 — Merge origin/LongNguyen trước

Lý do đi trước: nhiều việc business-logic hơn nhưng không có blocker kỹ thuật cứng như permissions.py của Minh Anh — thuận tay hơn để làm trước.

- Resolve `worktracker_core/urls.py`: thêm route Manager mới (dùng đúng quy ước prefix đã/sẽ thống nhất — 2.5).
- Resolve 3 file `timesheets/urls_manager.py / views_manager.py / serializers_manager.py`: lấy nguyên bản Long.
- `INSTALLED_APPS`: thêm 'reports', 'projects', 'tasks', 'system' nếu team đã chốt dùng bộ app này (2.6).

Bước 2 — Merge origin/MinhAnh sau

- Resolve `accounts/permissions.py`: dùng bản Tuấn Tú, không thương lượng (2.3, lý do kỹ thuật).
- Resolve `accounts/urls.py + views.py`: nếu Minh Anh đã tự chuyển code sang `_admin.py` trước khi merge (khuyến nghị — báo trước để cô ấy làm), bước này gần như tự động sạch. Nếu chưa, transplant thủ công logic nghiệp vụ của cô ấy sang `views_admin.py/urls_admin.py`, đổi cách dùng `HasPermission`.
- Resolve `worktracker_core/urls.py` lần 2: thêm route Admin.

Bước 3 — Kiểm tra sau merge

1. `pip install -r requirements.txt`
2. `python manage.py check`
3. `python manage.py makemigrations --check --dry-run` (bắt lỗi trước khi migrate thật — đặc biệt quan trọng vì đã đổi migration accounts)
4. `python manage.py migrate`
5. Test thủ công bằng `curl`: `POST /api/auth/login/` (route gốc còn sống), route Manager mới (Long), route Admin mới (Minh Anh) — mỗi bên 1-2 endpoint đại diện.

Phần 5 — Việc cần cả 3 người quyết định cùng nhau (đã dedup, xếp theo độ ưu tiên)

1. **[Cao nhất]** `accounts/permissions.py` — Minh Anh cần biết ngay: bản của cô ấy không dùng được, cần viết class mới nếu cần tính năng riêng.
2. **[Cao]** Kiến trúc app cuối cùng — dùng `projects/system/reports` (Long + Minh Anh) làm nền chính thức, bỏ scaffold rỗng `clients/jobs/audit` bên Tuấn Tú? (chi phí thấp vì scaffold đang rỗng — 2.6)
3. **[Cao]** Quy ước tiền tố route thống nhất — hiện có 3 kiểu khác nhau (2.5), càng để lâu càng khó đổi vì Frontend sẽ bắt đầu gọi theo 1 trong 3 kiểu.
4. **[Trung bình]** Ai giữ quyền sở hữu `timesheets` sau merge — Long giữ Manager, Tuấn Tú giữ Employee, cần ranh giới rõ để không dẫm chân nhau.
5. **[Trung bình]** Buổi nói riêng với Minh Anh về `accounts/models.py` — tuy đã nhẹ hơn tưởng (2.2), vẫn cần cô ấy xác nhận đồng ý dùng chung 1 migration mới, không tự sửa `0002/0003/0004` nữa.
6. **[Thấp]** `DATABASES` full-env config của Minh Anh — cài tiến độc lập, không chặn merge, đáng áp dụng bất kể kết quả các mục trên.

Trạng thái nhánh/worktree test (tham khảo, không phải để merge thật)

- `test_merge1` (local, tách từ TuanTu) — merge origin/LongNguyen, dở dang, chưa commit.
- `test_merge_minhanh` (local, tách từ TuanTu) — merge origin/MinhAnh trong 1 git worktree riêng, dở dang, chưa commit.

Cả 2 chỉ dùng để đối chiếu nội dung, không phải nhánh merge thật. Khi bắt tay làm thật theo Phần 4, nên tạo nhánh mới từ TuanTu mới nhất (sau khi đã đồng bộ với team về Phần 5), làm Bước 0 trước, rồi mới merge lần lượt.